

Projektarbeit

LEGO::Lab: Schnittstellen zu externen Displays

im Studiengang
Informatik Master

vorgelegt von

Florian Kaiser
Matrikelnummer: 79728
kaf11018@h-ka.de

an der Hochschule Karlsruhe
am 07.08.2026

Prüfer: Uwe Haneke

ERKLÄRUNG

Hiermit erkläre ich, dass ich die vorliegende Arbeit eigenständig und ohne fremde Hilfe angefertigt habe. Textpassagen, die wörtlich oder dem Sinn nach auf Publikationen oder Vorträgen anderer Autoren beruhen, sind als solche kenntlich gemacht. Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Karlsruhe, 07.08.2026

Florian Kaiser

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problemstellung	1
1.2	Zielsetzung der Arbeit	1
1.3	Aufbau der Arbeit	2
2	Grundlagen	2
2.1	Eingebettete Systeme und ressourcenbeschränkte Laufzeitumgebungen	3
2.2	Kommunikation in eingebetteten Systemen	3
2.3	Serialisierung von Nachrichten	3
2.4	Remote Procedure Call	3
2.5	Objektorientierte Abstraktion verteilter Systeme	3
2.6	Middleware als Architekturkonzept	3
2.7	Der betrachtete Bus und seine Transportbibliothek	3
3	Verwandte Arbeiten	3
3.1	Einordnung und Auswahlkriterien	4
3.2	Etablierte Rahmenwerke für entfernte Aufrufe	4
3.3	Serialisierung für ressourcenarme Geräte	5
3.4	Protokolle für eingebettete Systeme und Sensornetze	5
3.5	PUPRemote als Transportbasis	6
3.6	Abgrenzung	6
4	Systemanalyse und Anforderungen	7
4.1	Zielsystem und Hardwareumgebung	8
4.1.1	Der LEGO Education SPIKE Prime Hub als Ausgangsplattform	8
4.1.2	Notwendigkeit einer externen Erweiterung	8
4.2	Anforderungen an die Display-Integration	9
4.2.1	Anforderungen an ein externes Display	9
4.2.2	Anforderungen an eine externe Steuerungseinheit	9
4.2.3	Resultierende Systemstruktur	9
4.3	Kommunikationsanforderungen	10
4.4	Einschränkungen der Plattform	10
4.4.1	Eingeschränkte Kommunikationsrichtung	10
4.4.2	Begrenzte Paketgröße	10
4.4.3	Beschränkte Laufzeitumgebung	10
4.5	Verallgemeinerung des Problemraums	10
5	Architektur des Systems	10
5.1	Zielsetzung der Architektur	11
5.2	Schichtenmodell	11

5.2.1	Transportschicht	12
5.2.2	Nachrichtenschicht	12
5.2.3	Aufrufschicht	12
5.2.4	Objektmodell	13
5.2.5	Anwendung und Anzeige	13
5.3	Rollen im Entwurf	13
5.3.1	Interceptor	13
5.3.2	Dispatcher	13
5.3.3	Stellvertreter und Adapter	14
5.4	Kommunikationsmodell	14
5.5	Datenfluss im System	14
6	Kommunikations- und Transportschicht	14
6.1	Motivation und Randbedingungen	15
6.2	Interceptor-Abstraktion	15
6.3	Anfragegetriebener Rückkanal	15
6.4	Frame-basiertes Nachrichtenprotokoll	15
6.4.1	Frame-Struktur	15
6.4.2	Nutzlastlänge und Formatunabhängigkeit	15
6.4.3	Kostenmodell der Rahmung	15
6.5	Fragmentierung und Rekonstruktion	15
6.6	Serialisierung der Nutzdaten	15
6.7	Zuverlässigkeitsmechanismen	15
7	Abstraktion des Displays und Objektmodell	15
7.1	Anforderungen an die Display-Abstraktion	16
7.2	Konzept der Remote-Objekte	16
7.3	Abbildung grafischer Elemente	16
7.4	Objekterzeugung, Referenzen und Lebenszyklus	16
7.5	Methodenaufrufe und Ereignisbehandlung	16
7.6	Designentscheidungen	16
8	Hardwareaufbau	16
8.1	Aufbau und Komponenten	17
8.2	Adapterplatine	17
8.2.1	Signalzuordnung und Pinbelegung	17
8.2.2	Aufbau und Inbetriebnahme	17
8.3	Integrierte Trägerplatine	17
9	Referenzimplementierung	17
9.1	Laufzeitumgebung	18
9.2	Anbindung des Displays	18
9.3	Bindung an den Bus	18
9.4	Datenstrukturen	18

9.4.1	Frame	18
9.4.2	Message	18
9.4.3	Payload	18
9.5	Ablauf eines vollständigen Aufrufs	18
9.6	Nebenläufigkeit und Fehlerbehandlung	18
10	Evaluation	18
10.1	Funktionaler Nachweis	19
10.2	Overhead durch Rahmung	19
10.3	Latenz und Anzahl der Übertragungen	19
10.4	Ressourcenverbrauch	19
10.5	Bewertung des Hardwareaufbaus	19
10.6	Diskussion der Ergebnisse	19
11	Diskussion und Übertragbarkeit	19
11.1	Limitierungen	20
11.1.1	Kommunikationsmodell	20
11.1.2	Latenz und Durchsatz	20
11.1.3	Abstraktionsgrenzen	20
11.2	Übertragbarkeit auf andere Verbindungsklassen	20
11.3	Mögliche Erweiterungen	20
11.3.1	Ereignisgetriebener Rückkanal	20
11.3.2	Alternative Serialisierung	20
11.3.3	Weitere Peripherieklassen	20
12	Fazit	20
13	Abbildungs- und Tabellenverzeichnis	22
14	Abkürzungsverzeichnis	23
	Bibliografie	24

1 Einleitung

Die Vermittlung von Informatik- und Technikenkenntnissen an Kinder und Jugendliche gewinnt zunehmend an Bedeutung. Plattformen wie die von LEGO entwickelten Education-Systeme bieten hierfür einen niedrighschwelligigen Einstieg in Programmierung und Robotik. Durch vereinfachte Hardware und abstrahierte Softwareumgebungen ermöglichen sie erste praktische Erfahrungen ohne umfangreiche Vorkenntnisse.

Mit zunehmender Nutzung entsteht jedoch häufig der Wunsch, solche Systeme über ihren ursprünglichen Funktionsumfang hinaus zu erweitern. Insbesondere die Integration zusätzlicher Hardwarekomponenten eröffnet neue Möglichkeiten, etwa im Bereich der Visualisierung und Interaktion. Displays mit Touch-Funktionalität stellen in diesem Zusammenhang eine naheliegende Erweiterung dar, da sie eine direkte und intuitive Benutzerinteraktion ermöglichen.

Die Erweiterung bestehender Embedded-Systeme um externe Komponenten ist jedoch nicht trivial. Neben der reinen Hardwareanbindung spielen insbesondere Aspekte der Kommunikation und der Softwareintegration eine entscheidende Rolle. Unterschiedliche Systemkomponenten müssen koordiniert zusammenarbeiten, wobei vorhandene Schnittstellen und Laufzeitumgebungen bestimmte Einschränkungen mit sich bringen können.

Vor diesem Hintergrund beschäftigt sich die vorliegende Arbeit mit der Frage, wie sich externe Peripherie an ein solches System anbinden lässt, wenn die verfügbare Verbindung dafür nur eingeschränkt geeignet ist. Der Schwerpunkt liegt dabei nicht auf der Ansteuerung eines einzelnen Geräts, sondern auf einer Softwareschicht, die zwischen Anwendung und Verbindung vermittelt und die Eigenheiten der Übertragung vor der Anwendung verbirgt. Die Anbindung eines TFT-Touchscreen-Displays dient als Anwendungsfall, an dem dieser Entwurf entwickelt, umgesetzt und bewertet wird.

1.1 Problemstellung

Die Anbindung zusätzlicher Peripherie an eine solche Plattform wird durch drei technische Einschränkungen erschwert, die gemeinsam auftreten und sich gegenseitig verstärken.

Erstens ist die Verbindung zwischen den Systemkomponenten anfragegetrieben. Sie ist zwar in beide Richtungen nutzbar, doch nur die übergeordnete Seite kann eine Übertragung anstoßen, allerdings kann die untergeordnete Seite antwortet ausschließlich. Zweitens ist die je Paket übertragbare Datenmenge sehr klein und fest vorgegeben, sodass Nachrichten üblicher Größe grundsätzlich nicht in ein einzelnes Paket passen. Drittens stehen auf beiden Seiten stark beschränkte Laufzeitumgebungen zur Verfügung, die weder einen vollständigen Netzwerkstack noch die üblichen Mittel nebenläufiger Programmierung bereitstellen.

Diese Rahmenbedingungen stehen im Gegensatz zu den Anforderungen interaktiver Benutzeroberflächen, die eine flexible Übertragung strukturierter Daten sowie die zeitnahe Rückmeldung von Benutzereingaben voraussetzen. Insbesondere die Verarbeitung von Eingaben und die Übertragung zusammengesetzter Zustände stellen unter diesen Bedingungen eine Herausforderung dar.

Daraus ergibt sich die zentrale Problemstellung dieser Arbeit: Wie lässt sich eine Middleware entwerfen, die entfernte Objekte über eine anfragegetriebene Verbindung mit sehr kleiner Paketgröße auf beschränkten Laufzeitumgebungen transparent nutzbar macht?

Diese Frage ist nicht an eine bestimmte Plattform gebunden. Die genannte Kombination aus anfragegetriebener Übertragung, kleiner Paketgröße und beschränkter Laufzeitumgebung tritt in verschiedenen Ausprägungen auf, wie in Abschnitt 4.5 ausgeführt wird.

1.2 Zielsetzung der Arbeit

Ziel dieser Arbeit ist der Entwurf einer Middleware, die entfernte Objekte über eine Verbindung der beschriebenen Art transparent nutzbar macht.

Der Entwurf soll die drei genannten Einschränkungen auf wenige Schichten eingrenzen, sodass sie oberhalb davon nicht mehr sichtbar sind. Dazu gehören:

- die Zerlegung von Nachrichten in Pakete und deren Rahmung,
- ein anfragegetriebener Rückkanal für Ergebnisse und Ereignisse
- sowie eine Abstraktion, die entfernte Objekte wie lokale erscheinen lässt.

Ein weiterer Schwerpunkt liegt darauf, den Entwurf vom konkreten Übertragungsweg zu lösen, sodass er auf andere Verbindungen derselben Klasse übertragbar bleibt.

Die Bewertung stützt sich auf drei Kriterien: den funktionalen Nachweis, dass sich eine interaktive Oberfläche über die entwickelte Abstraktion umsetzen lässt, den Übertragungsaufwand je Aufruf, gemessen in benötigten Paketen und in der resultierenden Verzögerung, sowie den Ressourcenverbrauch auf beiden Seiten der Verbindung.

1.3 Aufbau der Arbeit

Die vorliegende Arbeit ist wie folgt aufgebaut:

2 Grundlagen

2.1 Eingebettete Systeme und ressourcenbeschränkte Laufzeitumgebungen

2.2 Kommunikation in eingebetteten Systemen

2.3 Serialisierung von Nachrichten

2.4 Remote Procedure Call

2.5 Objektorientierte Abstraktion verteilter Systeme

2.6 Middleware als Architekturkonzept

2.7 Der betrachtete Bus und seine Transportbibliothek

3 Verwandte Arbeiten

Die in dieser Arbeit entwickelte Middleware steht in einer langen Tradition von Ansätzen für entfernte Aufrufe und für die Kommunikation in verteilten und eingebetteten Systemen. Dieses Kapitel ordnet die eigene Lösung in dieses Umfeld ein. Da das Feld sehr breit ist, werden ausgewählte Vertreter betrachtet und entlang einheitlicher Kriterien verglichen. Ziel ist es, die bestehende Lücke zu benennen, die die vorliegende Arbeit schließt.

3.1 Einordnung und Auswahlkriterien

Für den Vergleich werden Kriterien herangezogen, die sich unmittelbar aus den Rahmenbedingungen des Zielsystems ergeben. Das Zielsystem nutzt einen Master-Slave-Bus mit sehr kleinen, festen Paketen und läuft in stark eingeschränkten Laufzeitumgebungen.

Folgende Achsen werden betrachtet:

- **Abstraktionsgrad.** Bietet der Ansatz nur den Transport von Nachrichten, einen entfernten Prozeduraufruf oder darüber hinaus die Abstraktion entfernter Objekte?
- **Serialisierung.** Werden Daten textbasiert oder binär kodiert, und ist dafür ein vorab geteiltes Schema nötig?
- **Transportbindung.** Ist der Ansatz an einen konkreten Übertragungsweg gebunden oder von diesem unabhängig?
- **Fragmentierung.** Bringt der Ansatz Mechanismen für Framing und das Zerlegen großer Nachrichten in kleine Pakete mit?
- **Eignung für einen Master-Slave-Bus.** Setzt der Ansatz voraus, dass die untergeordnete Seite eigenständig senden kann, oder kommt er mit einem rein anfragegetriebenen Kanal aus?
- **Ressourcenbedarf.** Lässt sich der Ansatz innerhalb der Grenzen von MicroPython und Pybricks betreiben?

Die betrachteten Ansätze liegen dabei nicht alle auf derselben Ebene. Einige dieser Rahmenwerke sind vollständig, während andere sich ausschließlich auf die Beschreibung eines Nachrichtenformats beschränken. Wieder andere fokussieren sich lediglich auf die Kodierung von Daten oder die Anbindung an einen bestimmten Bus. Ein Vergleich, der diese Unterschiede außer Acht lässt, stellt Sachverhalte nebeneinander, die sich gegenseitig gar nicht ersetzen können. Gemäß dem abschließenden Vergleich wird die Ebene explizit als separate Spalte ausgewiesen.

Diese Kriterien strukturieren die folgende Betrachtung und münden in den abschließenden Vergleich in Tabelle I.

3.2 Etablierte Rahmenwerke für entfernte Aufrufe

Das Konzept von entfernten Prozeduraufrufen wurde früh standardisiert. Der [Open Network Computing Remote Procedure Call \(ONC RPC\)](#) [1], [2] legt ein Protokoll und mit der External Data Representation ein eigenes Serialisierungsformat fest [3]. Der Ansatz ist auf leistungsfähigere Netzwerke ausgelegt und nicht für Kanäle mit Paketgrößen im Bereich weniger Byte gedacht.

JSON-RPC verfolgt eine deutlich schlankere Idee [4]. Die Spezifikation beschreibt ausschließlich das Format der Anfrage- und Antwortnachrichten auf Basis der [JavaScript Object Notation \(JSON\)](#) und überlässt den Transport bewusst der Anwendung. In ihrer Philosophie ist diese Trennung dem hier verfolgten Ansatz nahe. Allerdings setzt JSON-RPC voraus, dass eine darunterliegende Schicht vollständige Nachrichten zustellen kann. Es definiert weder ein Framing noch eine Fragmentierung für sehr kleine Pakete und sieht keinen Mechanismus für einen anfragegetriebenen Rückkanal auf einem Master-Slave-Bus vor.

Am anderen Ende des Spektrums steht [gRPC Remote Procedure Call \(gRPC\)](#) [5]. Das Rahmenwerk verbindet Protocol Buffers als schemagebundene Serialisierung mit HTTP/2 als Transport und unterstützt unter anderem Datenströme in beide Richtungen. Diese Leistungsfähigkeit erfordert jedoch einen vollständigen HTTP/2-Stack und setzt eine bidirektionale

Verbindung voraus. Für den betrachteten LEGO-Bus und die eingesetzten Laufzeitumgebungen ist [gRPC](#) zu schwergewichtig und zu stark an seinen Transport gebunden.

Zusammengefasst setzen die etablierten Rahmenwerke entweder einen leistungsfähigen Transport voraus oder standardisieren nur die Nachrichtenebene und lassen Framing, Fragmentierung und den Rückkanal offen. Genau diese darunterliegende Schicht adressiert die vorliegende Arbeit.

3.3 Serialisierung für ressourcenarme Geräte

Da die Serialisierung die Nachrichtengröße und damit die Anzahl der nötigen Pakete bestimmt, ist sie für ein System mit kleinen Paketen besonders relevant. Die [JSON](#) ist schemafrei und gut lesbar, erzeugt aber durch Klartextschlüssel und Strukturzeichen vergleichsweise große Ausgaben [6]. Binäre und schemafreie Formate wie [MessagePack](#) [7] und die [Concise Binary Object Representation \(CBOR\)](#) [8], [9] erreichen bei gleichem Datenmodell deutlich kompaktere Darstellungen. Schemagebundene Formate wie [Protocol Buffers](#) gehen noch einen Schritt weiter und setzen eine geteilte Beschreibung der Datenstruktur voraus [10]. Für sehr kleine Geräte existiert mit [Nanopb](#) [11] eine ressourcenschonende Implementierung dieses Ansatzes.

Die vorliegende Arbeit verwendet zunächst die [JSON](#). Ausschlaggebend dafür waren zwei praktische Gründe. Zum einen steht das Format in beiden Laufzeitumgebungen ohne zusätzliche Abhängigkeit zur Verfügung. Zum anderen ist die Nutzlast im Klartext lesbar, was auf einem Bus, dessen Datenverkehr sich nur mit erheblichem Aufwand mitschneiden lässt, einen spürbaren Vorteil bei der Fehlersuche darstellt. Solange die übrigen Schichten noch im Entstehen waren, wog diese Lesbarkeit schwerer als die Kompaktheit der Kodierung.

Mit zunehmender Reife der Umsetzung kehrte sich dieses Verhältnis um. Bei einer Nutzlast von wenigen Byte je Paket schlägt jedes zusätzliche Zeichen unmittelbar auf die Anzahl der zu übertragenden Pakete und damit auf die Dauer eines Aufrufs durch. Die Arbeit wechselt daher im weiteren Verlauf auf [MessagePack](#) [7]. Das Format bildet dasselbe Datenmodell ab, kommt ebenfalls ohne ein vorab geteiltes Schema aus und stellt es binär und damit deutlich kürzer dar. Die Lesbarkeit im Fehlerfall geht dabei verloren, lässt sich jedoch durch ein Umschalten auf die textbasierte Kodierung zurückgewinnen, da beide Formate hinter derselben Schnittstelle liegen.

Das Umwandeln und das Auslesen der Nutzlast sind deshalb an einer einzigen Stelle gebündelt und nicht über die Schichten verteilt. Das Format ist damit ein Variationspunkt: Ein Wechsel der Kodierung betrifft diese Stelle sowie die Rahmung, die dafür eine ausdrückliche Längenangabe je Paket benötigt, nicht aber den entfernten Aufruf oder das Objektmodell. Der beschriebene Wechsel ist zugleich der Nachweis, dass diese Trennung hält. Die Evaluation vergleicht beide Kodierungen und beziffert den Overhead nicht nur, sondern weist ihn in benötigten Paketen je Aufruf aus.

3.4 Protokolle für eingebettete Systeme und Sensornetze

Neben den allgemeinen Rahmenwerken existieren Protokolle, die näher an der betrachteten Domäne liegen. [Firmata](#) [12] ermöglicht die Steuerung eines Mikrocontrollers von einem übergeordneten Rechner aus und überträgt dazu einen festgelegten Satz von Befehlen über eine serielle Verbindung. Die Asymmetrie zwischen einem treibenden Rechner und einem reagierenden Gerät ähnelt der hier vorliegenden Situation. Allerdings beschränkt sich [Firmata](#) auf einen vordefinierten Befehlsumfang und bietet weder einen allgemeinen entfernten Aufruf noch eine Abstraktion entfernter Objekte.

[MQTT](#) [13] ist auf ressourcenarme Geräte und kleine Pakete ausgelegt und kommt ohne eine durchgehende TCP-Verbindung aus. Es folgt jedoch einem Modell aus Veröffentlichen und Abonnieren, benötigt ein Gateway sowie einen Vermittler und setzt voraus, dass das Gerät eigenständig veröffentlichen kann. Auf einem reinen Master-Slave-Bus, auf dem die untergeordnete Seite nicht von sich aus senden darf, ist dieses Modell nicht unmittelbar anwendbar.

Diese Protokolle sind der betrachteten Domäne nahe, lösen aber eine andere Aufgabe. Sie zielen auf Gerätesteuerung mit festem Befehlssatz oder auf nachrichtenbasiertes Veröffentlichen und Abonnieren und nicht auf einen allgemeinen, objektorientierten entfernten Aufruf über einen anfragegetriebenen Kanal.

3.5 PUPRemote als Transportbasis

PUPRemote [14] nimmt in diesem Umfeld eine Sonderstellung ein, da es nicht als Alternative, sondern als Baustein der eigenen Lösung auftritt. Die Bibliothek erschließt den PUP- und LPF2-Bus und stellt darauf benannte Aufrufe bereit. Jeder Aufruf belegt einen eigenen Modus des Busses und ist über feste Formatangaben für beide Richtungen beschrieben. Der Hub schreibt die Argumente in den zugehörigen Modus und liest anschließend den Rückgabewert aus demselben Modus zurück. Mittels PUPRemote werden beidseitig Befehle registriert, welche dann durch den Master aufgerufen werden können. Jedoch sind Länge der Methodennamen, Argumente und Rückgabewerte durch feste Formatangaben begrenzt. Damit besitzt PUPRemote durchaus eine Aufrufsemantik und ist nicht auf reinen Datentransport beschränkt.

Diese Semantik trägt jedoch nur so weit, wie sich ein Aufruf auf einen vorab registrierten Modus und ein einzelnes Paket fester Größe abbilden lässt. Die Anzahl der Modi ist durch den Bus begrenzt, die Formate stehen zur Übersetzungszeit fest, und für Nachrichten, die größer sind als ein Paket, ist kein Mechanismus vorgesehen. Eine Middleware, die beliebige Methoden auf einer zur Laufzeit wachsenden Menge entfernter Objekte anbieten soll, kann daher nicht einen Modus je Aufruf vergeben.

Die vorliegende Arbeit nutzt PUPRemote deshalb nicht als Aufrufmechanismus, sondern als Transportbindung. Sie registriert lediglich zwei Kommandos für den Hin- und den Rückweg, deren Nutzlast aus Sicht von PUPRemote ein undurchsichtiger Block fester Länge ist. Die Zerlegung von Nachrichten in Pakete, deren Rahmung, die Zuordnung von Antworten zu Anfragen, der client-gesteuerte Rückkanal sowie das Objektmodell aus Stellvertretern und serverseitigen Adaptern werden vollständig oberhalb davon angesiedelt. PUPRemote bleibt damit austauschbar und erfüllt die Rolle einer konkreten Transportschicht und kann durch andere Protokolle ersetzt werden, die dieselbe Anforderung erfüllen.

3.6 Abgrenzung

Tabelle I fasst die betrachteten Ansätze entlang der eingangs definierten Kriterien zusammen.

Tabelle I: Vergleich der betrachteten Ansätze entlang der Auswahlkriterien

Ansatz	Ebene	Abstraktionsgrad	Serialisierung	Transportbindung	Fragmentierung	Eignung kleiner Master-Slave-Bus
gRPC	Middleware	RPC und Datenströme	Protocol Buffers, schemagebunden	an HTTP/2 gebunden	über HTTP/2	ungeeignet, setzt Push voraus
JSON-RPC	Nachrichtenformat	nur Nachrichtenformat	JSON, textbasiert	unabhängig, offen	nicht definiert	offen, Rückkanal fehlt
Message-Pack und CBOR	Serialisierung	nur Serialisierung	binär, schemafrei	entfällt	entfällt	entfällt
Firmata	Geräteprotokoll	Gerätesteuerung mit festen Befehlen	binär, fest	seriell, fest	nein	passend, aber fester Befehlssatz
MQTT	Nachrichtenprotokoll	Veröffentlichen und Abonnieren	binär, kompakt	unabhängig, braucht Gateway	teilweise	setzt eigenständiges Senden voraus
PUPRemote	Transportanbindung, hier als Baustein verwendet	entfernter Aufruf, ein Modus je Aufruf	feste Formatangaben je Modus	an PUP und LPF2 gebunden	nein, ein Paket	passend, aber nur ein Paket
Diese Arbeit	Middleware	entfernter Aufruf und entfernte Objekte	JSON, gekapselt und austauschbar	gekapselt, PUPRemote als erste Bindung	ja, eingebaut	passend, clientgesteuerter Rückkanal

Aus dem Vergleich wird die Lücke deutlich. Die leistungsfähigen Rahmenwerke setzen einen Transport voraus, den das Zielsystem nicht bietet. Die schlanken Ansätze standardisieren nur Teilaspekte und lassen Framing, Fragmentierung und den Rückkanal offen. Die domänennahen Protokolle lösen eine andere Aufgabe oder setzen voraus, dass die untergeordnete Seite eigenständig senden kann. PUPRemote schließlich liegt eine Ebene tiefer: Es erschließt den Bus und wird in dieser Arbeit als Transportbindung genutzt, tritt damit aber nicht an die Stelle der gesuchten Middleware.

Die vorliegende Arbeit verbindet diese Aspekte zu einer zusammenhängenden Lösung. Sie stellt einen transportunabhängig konzipierten entfernten Aufruf samt Objektmodell bereit, bringt Framing und Fragmentierung für sehr kleine Pakete mit und realisiert einen clientgesteuerten Rückkanal für einen Master-Slave-Bus, und das innerhalb der Grenzen von MicroPython und Pybricks. Der Beitrag liegt weniger in einem einzelnen neuen Grundbaustein als in der stimmigen Verbindung bekannter Konzepte für einen bislang nicht abgedeckten Anwendungsfall.

4 Systemanalyse und Anforderungen

4.1 Zielsystem und Hardwareumgebung

Ausgangspunkt der vorliegenden Arbeit ist eine bestehende Hardwareplattform aus dem LEGO-Education-Ökosystem. Die Plattform definiert die technischen Rahmenbedingungen, innerhalb derer die in Abschnitt 1 motivierte Display-Integration realisiert werden muss. Im Folgenden wird zunächst die zentrale Hardwarekomponente vorgestellt, anschließend werden ihre Eigenschaften im Hinblick auf die Anbindung eines Touchscreen-Displays analysiert. Aus dieser Analyse heraus ergibt sich die Notwendigkeit einer externen Erweiterung, deren mögliche Ausprägungen im letzten Abschnitt skizziert werden.

4.1.1 Der LEGO Education SPIKE Prime Hub als Ausgangsplattform

Den Kern des Zielsystems bildet der LEGO Education SPIKE Prime Hub [15]. Hierbei handelt es sich um einen programmierbaren Steuerbaustein, der primär für den Einsatz im Bildungsbereich konzipiert ist und über sechs Ein- und Ausgabeports zum Anschluss von LEGO-kompatiblen Sensoren und Aktoren verfügt. Zur Interaktion mit dem Nutzer stellt der Hub eine 5×5-LED-Matrix, drei Tasten sowie einen einfachen Lautsprecher bereit.

Intern basiert der Hub auf einem STM32F413-Mikrocontroller [16] mit einem ARM-Cortex-M4-Kern, einer Taktfrequenz von etwa 100 MHz, rund 1 bis 1,5 MB Flash-Speicher sowie 320 KB SRAM [17]. Die Programmierung kann grafisch über die offizielle LEGO-Software oder textbasiert mittels MicroPython erfolgen. Welche Firmware konkret zum Einsatz kommt und welche Auswirkungen diese Wahl auf die spätere Lösung hat, wird in Abschnitt 4.4 vertieft.

Für die vorliegende Arbeit sind drei Eigenschaften des Hubs von besonderer Bedeutung:

- Der Hub stellt ausschließlich passive, an die Sensor- und Motorports gebundene Erweiterungsmöglichkeiten bereit. Eine direkte Anbindung beliebiger Peripheriegeräte über offene Schnittstellen wie SPI [18] oder I2C [19] ist nicht vorgesehen.
- Die verfügbaren Anzeige- und Eingabemöglichkeiten beschränken sich auf die genannte LED-Matrix sowie die Hub-Tasten.
- Die Rechenleistung und der Arbeitsspeicher sind für einen Mikrocontroller zwar typisch, im Hinblick auf grafische Benutzeroberflächen jedoch stark limitiert.

Diese Eigenschaften legen den Rahmen fest, innerhalb dessen die Display-Integration konzipiert werden muss.

4.1.2 Notwendigkeit einer externen Erweiterung

Aus den in Kapitel 1 motivierten Anforderungen an eine grafische Benutzeroberfläche mit Touch-Funktionalität ergibt sich unmittelbar ein Konflikt mit der beschriebenen Plattform. Die im Hub integrierten Ausgabe- und Eingabemöglichkeiten beschränken sich auf eine niedrig auflösende LED-Matrix und einfache Tasten und sind damit für die Darstellung komplexer Inhalte sowie für eine differenzierte Benutzerinteraktion ungeeignet. Auch eine Erweiterung des Hubs selbst ist nicht vorgesehen, da der Hub als geschlossene Komponente konzipiert ist und keine Schnittstellen zum direkten Anschluss grafischer Peripheriegeräte bereitstellt.

Daraus folgt, dass die geforderte Display-Funktionalität nicht innerhalb des Hubs, sondern ausschließlich außerhalb davon realisiert werden kann. Die Aufgabe der Display-Ansteuerung muss demnach an eine externe Hardwarekomponente delegiert werden, die ihrerseits an einen der vorhandenen Sensorports des Hubs angebunden wird. Hieraus ergeben sich zwei eigenständige Teilprobleme, die in den folgenden Abschnitten weiter ausgearbeitet werden:

- die Wahl einer geeigneten externen Hardware zur Ansteuerung eines Displays sowie
- die Etablierung einer geeigneten Kommunikationsbeziehung zwischen Hub und dieser externen Komponente.

4.2 Anforderungen an die Display-Integration

4.2.1 Anforderungen an ein externes Display

Bevor konkrete Hardwarekomponenten betrachtet werden, lassen sich aus der Zielsetzung der Arbeit allgemeine Anforderungen an ein externes Display ableiten. Das Display soll die Darstellung einfacher grafischer Elemente wie Schaltflächen, Beschriftungen und Anzeigen ermöglichen und zugleich Benutzereingaben über einen Touchscreen entgegennehmen. Hinsichtlich Auflösung, Größe und Energieaufnahme orientieren sich die Anforderungen an typischen Embedded-Anwendungen und liegen damit im Bereich kleiner TFT-Displays mit Auflösungen im Bereich von etwa 240×320 Pixeln.

Solche Displays sind in unterschiedlichen Ausführungen verfügbar. Eine im Embedded-Bereich weit verbreitete Variante stellen Module mit ILI9341-Display-Controller dar, die eine Auflösung von 240×320 Pixeln bei einer Farbtiefe von 18 Bit unterstützen und über die SPI-Schnittstelle angesteuert werden können [ili9341_datasheet]. Für die Erkennung von Berührungen kommen typischerweise kapazitive Touch-Controller wie der FT6X36 zum Einsatz, die über I2C angebunden werden und neben Einzelberührungen auch Mehrfingergesten erfassen können [ft6x36_arduino_forum]. Solche Module sind kostengünstig verfügbar und werden in der Maker- und Embedded-Community häufig in Verbindung mit Mikrocontrollern der ESP32-Familie eingesetzt.

4.2.2 Anforderungen an eine externe Steuerungseinheit

Da der SPIKE Prime Hub selbst keine direkte Anbindung eines TFT-Displays über SPI ermöglicht, muss zwischen Hub und Display eine zusätzliche Steuerungseinheit vermitteln. Diese übernimmt die unmittelbare Ansteuerung des Displays sowie die Verarbeitung der Touch-Eingaben und stellt gleichzeitig die Kommunikation mit dem Hub sicher. Aus dieser Doppelrolle ergeben sich mehrere Anforderungen an die einzusetzende Hardware:

- Sie muss über geeignete Schnittstellen zur Ansteuerung typischer TFT-Displays verfügen, insbesondere SPI für die Bildschirmdaten und I2C für den Touch-Controller.
- Sie muss ausreichend Rechenleistung und Speicher bieten, um eine grafische Bibliothek wie etwa LVGL (Light and Versatile Graphics Library) ausführen zu können.
- Sie muss über eine Schnittstelle verfügen, die kompatibel zu den Sensorports des SPIKE Prime Hubs ist, oder eine entsprechende Anbindung über das LEGO-Powered-Up-UART-Protokoll (LPF2) unterstützen.

Diese Anforderungen werden von im Bildungs- und Maker-Bereich verfügbaren Erweiterungsboards erfüllt, die speziell für die Anbindung an LEGO-Hubs entworfen wurden. Ein Beispiel hierfür sind ESP32-basierte Boards der LMS-ESP32-Reihe [antonsmindstorms_lms_esp32_product], die sich gegenüber dem Hub als LEGO-Sensor ausgeben und damit eine Kommunikation über das LPF2-Protokoll ermöglichen. Welche konkrete Hardware im Rahmen dieser Arbeit eingesetzt wird, sowie die zugehörige Softwarekonfiguration werden im Kapitel zur Referenzimplementierung dargestellt.

4.2.3 Resultierende Systemstruktur

Aus den genannten Überlegungen ergibt sich eine grundsätzliche Systemstruktur, die sich aus drei Teilen zusammensetzt: dem SPIKE Prime Hub als zentraler Steuereinheit, einer externen Steuerungseinheit zur unmittelbaren Ansteuerung des Displays sowie einem TFT-Touchscreen-Display als Anzeige- und Eingabeelement. Die Kopplung zwischen Hub und externer Steuerungseinheit erfolgt dabei über die vom Hub bereitgestellten Sensorports und das zugehörige LPF2-Protokoll. Die konkrete Ausgestaltung dieser Kommunikation und der damit verbundenen Einschränkungen wird in den folgenden Abschnitten dieses Kapitels detailliert betrachtet.

4.3 Kommunikationsanforderungen

4.4 Einschränkungen der Plattform

4.4.1 Eingeschränkte Kommunikationsrichtung

4.4.2 Begrenzte Paketgröße

4.4.3 Beschränkte Laufzeitumgebung

4.5 Verallgemeinerung des Problemraums

Die im vorigen Abschnitt herausgearbeiteten Einschränkungen wurden am konkreten Zielsystem hergeleitet. Sie sind jedoch nicht an dieses System gebunden, und diese Feststellung ist für die weitere Arbeit von Bedeutung: Sie entscheidet darüber, ob der folgende Entwurf eine Einzellösung darstellt oder eine Klasse von Verbindungen adressiert.

Die drei Eigenschaften lassen sich von der Plattform ablösen und als allgemeine Merkmale einer Verbindung formulieren. Erstens ist die Übertragung anfragegetrieben, das heißt, nur eine der beiden Seiten kann eine Übertragung anstoßen. Zweitens ist die Nutzlast je Übertragung klein und fest vorgegeben. Drittens steht auf mindestens einer Seite eine Laufzeitumgebung zur Verfügung, die weder einen vollständigen Netzwerkstack noch die üblichen Mittel nebenläufiger Programmierung bereitstellt.

Diese Kombination tritt über den betrachteten Bus hinaus in verschiedenen Ausprägungen auf. Bei der Kommunikation über I²C treibt der Master den Bus, und ein Slave kann eine Übertragung nicht selbstständig beginnen. Modbus RTU folgt demselben Muster aus Anfrage und Antwort. Auch bei Bluetooth Low Energy ist der verbreitete Zugriff über lesende Anfragen des Clients organisiert, und die standardmäßig ausgehandelte Nutzlast liegt in derselben Größenordnung wie beim hier betrachteten Bus. In allen genannten Fällen stellen sich dieselben Fragen: Wie werden Nachrichten zerlegt und wieder zusammengesetzt, wie gelangen Ergebnisse und Ereignisse zur anfragenden Seite zurück, und wie lässt sich all das vor der Anwendung verbergen.

Die vorliegende Arbeit behandelt den betrachteten Bus daher als einen Vertreter dieser Klasse. Der in den folgenden Kapiteln entwickelte Entwurf wird so formuliert, dass er auf die genannten Merkmale Bezug nimmt und nicht auf Eigenschaften einer bestimmten Plattform. Inwieweit dies gelungen ist, greift die Diskussion am Ende der Arbeit erneut auf.

5 Architektur des Systems

Dieses Kapitel entwickelt die Architektur der Lösung aus den Anforderungen des vorigen Kapitels. Die Darstellung bleibt bewusst auf der konzeptionellen Ebene und benennt weder konkrete Bibliotheken noch Datenformate oder Klassen. Beschrieben werden die Entwurfsziele, der geschichtete Aufbau mit den Aufgaben der einzelnen Schichten, die wiederkehrenden Entwurfsrollen innerhalb dieser Schichten sowie das Kommunikationsmodell. Wie sich dieser Entwurf auf eine bestimmte Hardware, eine bestimmte Laufzeitumgebung und einen bestimmten Bus abbilden lässt, zeigt das Implementierungskapitel anhand des im Rahmen dieser Arbeit umgesetzten Systems.

5.1 Zielsetzung der Architektur

Die Architektur verfolgt mehrere Ziele, die sich unmittelbar aus den zuvor formulierten Anforderungen ableiten. Im Mittelpunkt steht die Aufruftransparenz auf der Anwendungsseite. Die Anwendungslogik soll mit gewöhnlichen Methodenaufrufen arbeiten und nichts über Pakete, Bestätigungen oder das Abholen von Ergebnissen wissen müssen.

Ein zweites Ziel ist die Unabhängigkeit vom konkreten Übertragungsweg. Die höheren Schichten sollen so entworfen sein, dass der Transport hinter einer einheitlichen Schnittstelle liegt und ausgetauscht werden kann. Hinzu kommen die Bewältigung der kleinen Paketgröße durch Rahmung und Fragmentierung, ein anfragegetriebener Rückkanal für Ergebnisse und Ereignisse sowie ein sparsamer Umgang mit Speicher und Rechenzeit. Schließlich soll die Architektur eine klare und symmetrische Struktur besitzen, in der beide Seiten dieselben Schnittstellen teilen.

Aus diesen Zielen ergibt sich ein geschichteter Aufbau nach dem Vorbild einer Middleware mit entferntem Aufruf. Jede Schicht kapselt eine klar umrissene Aufgabe und stellt der darüberliegenden Schicht eine höherwertige Abstraktion bereit. Der Gewinn dieses Aufbaus liegt darin, dass sich die schwierigen Eigenschaften des Zielsystems, die geringe Paketgröße und die eingeschränkte Kommunikationsrichtung, auf wenige Schichten eingrenzen lassen. Oberhalb dieser Schichten sind sie nicht mehr sichtbar.

5.2 Schichtenmodell

Das System ist in fünf Schichten gegliedert, die auf beiden Seiten der Verbindung vorhanden sind. Abbildung 1 zeigt den Aufbau.



Abbildung 1: Schichtenmodell der Architektur. Beide Seiten sind gleich aufgebaut und verkehren jeweils nur mit der unmittelbar darunterliegenden Schicht.

Zwei Eigenschaften dieses Aufbaus sind hervorzuheben. Zum einen ist er symmetrisch: Beide Seiten besitzen dieselben Schichten mit denselben Verantwortlichkeiten, sie unterscheiden sich lediglich in der Richtung, aus der ein Aufruf eintrifft. Zum anderen ist er streng geschichtet. Jede Schicht spricht ausschließlich mit der unmittelbar darunterliegenden, sodass eine Änderung innerhalb einer Schicht die übrigen nicht berührt.

Die folgenden Abschnitte beschreiben die Schichten von unten nach oben. Für jede Schicht werden ihre Aufgabe, die von ihr nach oben angebotene Abstraktion und die bewusst nicht von ihr übernommenen Aufgaben benannt.

5.2.1 Transportschicht

Die Transportschicht ist die einzige Schicht mit Kenntnis des tatsächlichen Übertragungswegs. Ihre Aufgabe besteht darin, einen Block fester Länge zur Gegenseite zu befördern und einen ebensolchen Block als Antwort entgegenzunehmen. Sie kennt weder den Inhalt dieses Blocks noch dessen Bedeutung.

Nach oben bietet sie damit eine denkbar schmale Abstraktion an: einen paketweisen, vom Client angestoßenen Austausch fester Größe. Diese Schmalheit ist beabsichtigt, denn sie ist der Preis dafür, dass die Schicht austauschbar bleibt. Jeder Übertragungsweg, der einen anfragegetriebenen Austausch von Blöcken fester Länge leisten kann, ist als Unterbau geeignet.

Nicht zu ihren Aufgaben gehören die Zerlegung größerer Nachrichten, die Zuordnung von Antworten zu Anfragen und jede Form der Auslegung des Inhalts. Die Paketgröße ist für diese Schicht eine gegebene Eigenschaft des Übertragungswegs und keine Größe, die sie beeinflusst.

5.2.2 Nachrichtenschicht

Die Nachrichtenschicht überbrückt den Abstand zwischen einer Nachricht beliebiger Länge und den kleinen Paketen des Übertragungswegs. Sie ist damit die Schicht, in der sich die zentrale Einschränkung des Zielsystems niederschlägt.

Ihre erste Aufgabe ist die Serialisierung: Eine strukturierte Nachricht wird in eine Folge von Bytes überführt und auf der Gegenseite wieder ausgelesen. Ihre zweite Aufgabe ist die Rahmung. Jedes Paket erhält Steuerinformationen, die es der Gegenseite erlauben, die ursprüngliche Nachricht wiederherzustellen: eine Kennung der Nachricht, zu der das Paket gehört, seine Position innerhalb dieser Nachricht, die Angabe, ob es das letzte Paket ist, und die Anzahl der tatsächlich genutzten Nutzbytes. Ihre dritte Aufgabe ist die Fragmentierung, also das Zerlegen einer Nachricht in eine Folge solcher Pakete und deren Zusammensetzen auf der Gegenseite.

Nach oben verbirgt diese Schicht die Paketgröße vollständig. Höhere Schichten arbeiten mit Nachrichten, deren Länge sie nicht zu beachten brauchen.

Zwei Entwurfsentscheidungen dieser Schicht verdienen Beachtung. Erstens ist die Rahmung vom Serialisierungsformat getrennt. Beide sind an je einer Stelle gebündelt, sodass sich das Format wechseln lässt, ohne die Rahmung anzutasten. Zweitens muss die Rahmung die Länge der Nutzlast ausdrücklich mitführen und darf sie nicht aus dem Inhalt ableiten, etwa durch das Entfernen von Füllbytes am Ende. Andernfalls schränkt die Rahmung die zulässigen Inhalte ein und bindet die Schicht an ein textbasiertes Format. Bei einer Nutzlast von wenigen Byte je Paket ist die Größe der Steuerinformationen zudem kein Randaspekt, sondern der wesentliche Kostenfaktor der gesamten Übertragung.

5.2.3 Aufrufschicht

Die Aufrufschicht stellt den entfernten Aufruf her. Auf der aufrufenden Seite bildet sie aus dem Namen einer Methode, deren Argumenten und der Angabe des Wirkungsbereichs eine Nachricht, übergibt diese nach unten und nimmt anschließend das Ergebnis entgegen. Auf der ausführenden Seite nimmt sie eine Nachricht entgegen, wählt den zuständigen Empfänger, ruft die benannte Methode auf und stellt deren Ergebnis zur Abholung bereit.

In dieser Schicht liegt außerdem die Behandlung des Rückkanals. Da die ausführende Seite nicht von sich aus senden kann, wird das Ergebnis nicht zugestellt, sondern bereitgehalten und auf Anfrage herausgegeben. Die Aufrufschicht verbirgt diesen Umstand: Nach oben erscheint ein Aufruf als gewöhnlicher, synchroner Methodenaufruf, der zurückkehrt, sobald das Ergebnis vorliegt.

Nicht zu ihren Aufgaben gehört die Kenntnis der aufgerufenen Objekte. Sie befördert einen benannten Aufruf zu einem benannten Empfänger, ohne zu wissen, was dieser darstellt.

5.2.4 Objektmodell

Das Objektmodell hebt den entfernten Aufruf auf die Ebene entfernter Objekte. Grafische Elemente wie Bildschirme, Beschriftungen und Schaltflächen werden als Objekte abgebildet, die auf der ausführenden Seite bestehen und auf der aufrufenden Seite durch Stellvertreter vertreten werden.

Grundlage ist eine gemeinsame abstrakte Schnittstelle je Objekttyp, die auf beiden Seiten vorliegt. Damit ein Stellvertreter ein bestimmtes entferntes Objekt anspricht, vergibt die ausführende Seite beim Erzeugen eines Objekts eine Referenz und verwaltet die Objekte in einer Registratur. Der Stellvertreter führt diese Referenz mit und übergibt sie bei jedem Aufruf. Auf diese Weise lassen sich beliebig viele Objekte erzeugen und gezielt ansprechen.

Nach oben bietet diese Schicht eine Sicht an, die sich von einer rein lokalen Programmierung nicht unterscheidet. Sie ist damit die Schicht, die das Ziel der Aufruftransparenz einlöst.

5.2.5 Anwendung und Anzeige

Die oberste Schicht ist auf beiden Seiten verschieden, da hier die eigentliche Aufgabe liegt. Auf der Steuereinheit ist es die Anwendung, die das Objektmodell nutzt und keinerlei Kenntnis der Kommunikation besitzt. Auf der Anzeigeeinheit ist es die tatsächliche Darstellung und die Entgegennahme von Eingaben, also die Ansteuerung der Anzeigehardware und die Erkennung von Berührungen.

Beide Seiten sind über die darunterliegenden Schichten vollständig entkoppelt. Ein Wechsel der Anzeigehardware berührt ausschließlich diese oberste Schicht auf der Anzeigeeinheit.

5.3 Rollen im Entwurf

Innerhalb der Schichten treten drei wiederkehrende Rollen auf. Sie sind als Entwurfsmuster zu verstehen und legen fest, welche Verantwortung an welcher Stelle liegt, nicht wie sie umzusetzen ist.

5.3.1 Interceptor

Der Interceptor bildet die Naht zwischen der Aufrufschicht und dem Transport. Ein gemeinsamer Vertrag legt fest, wie ein Aufruf abgesetzt, eine Bestätigung verarbeitet und die Bereitschaft eines Ergebnisses geprüft wird. Auf der aufrufenden Seite verpackt der Interceptor einen Methodenaufruf, sendet dessen Pakete und holt anschließend das Ergebnis ab. Auf der ausführenden Seite nimmt er die Pakete entgegen, setzt die Nachricht zusammen und stellt das Ergebnis zur Abholung bereit.

Da dieser Vertrag unabhängig vom konkreten Übertragungsweg formuliert ist, bildet der Interceptor den Ansatzpunkt für die Transportunabhängigkeit. Ein anderer Übertragungsweg wird angebunden, indem eine weitere Ausprägung dieser Rolle bereitgestellt wird; die höheren Schichten bleiben unberührt.

5.3.2 Dispatcher

Der Dispatcher ordnet eine eingehende Nachricht der auszuführenden Methode zu. Jede Nachricht führt dazu die Angabe ihres Wirkungsbereichs, den Namen der Methode und deren Argumente mit sich. Anhand des Wirkungsbereichs wählt der Dispatcher den zuständigen Empfänger und ruft dort die benannte Methode auf.

Für jeden Objekttyp kann ein eigener Empfänger registriert werden. Diese Trennung entkoppelt das Routing der Nachrichten von der eigentlichen Umsetzung der Methoden und

erlaubt es, das Objektmodell schrittweise zu erweitern, ohne die darunterliegenden Schichten zu berühren.

5.3.3 Stellvertreter und Adapter

Stellvertreter und Adapter sind die beiden Seiten desselben Objekttyps. Der Stellvertreter implementiert die gemeinsame Schnittstelle auf der aufrufenden Seite und leitet jeden Methodenaufruf weiter. Der Adapter implementiert dieselbe Schnittstelle auf der ausführenden Seite und enthält die reale Logik, die der Dispatcher aufruft.

Dass beide dieselbe Schnittstelle erfüllen, ist die strukturelle Grundlage der Aufrufstransparenz: Für die Anwendung ist nicht erkennbar und nicht erheblich, ob sie mit einem Stellvertreter oder mit einer lokalen Umsetzung arbeitet.

5.4 Kommunikationsmodell

Aus Sicht der Anwendung ist die Kommunikation synchron. Ein Aufruf kehrt erst zurück, wenn das Ergebnis vorliegt. Unter dieser Oberfläche folgt die Übertragung dem anfragegetriebenen Muster des Übertragungswegs. Jede Übertragung wird von der Steuereinheit angestoßen, die Anzeigeeinheit antwortet nur.

Logisch werden zwei Kanäle unterschieden. Über den ersten Kanal sendet die Steuereinheit die Pakete eines Aufrufs, und die Anzeigeeinheit bestätigt jedes Paket. Über den zweiten Kanal fragt die Steuereinheit den Zustand und das Ergebnis ab. Sie prüft zunächst, ob ein Ergebnis bereitsteht, und holt es anschließend Paket für Paket ab.

Ereignisse wie Berührungen werden nach demselben Prinzip behandelt. Die Anzeigeeinheit puffert sie, und die Steuereinheit fragt sie bei Bedarf ab. Damit ist auch die Ereignisbehandlung an den Takt der Steuereinheit gebunden, was die zeitliche Auflösung begrenzt. Dieser client-gesteuerte Rückkanal ist die zentrale Antwort der Architektur auf die eingeschränkte Kommunikationsrichtung.

5.5 Datenfluss im System

Der Weg eines Aufrufs durch die Schichten lässt sich in vier Phasen gliedern. Zunächst nimmt der Stellvertreter den Methodenaufruf entgegen und übergibt ihn der Aufrufschicht. Diese bildet aus Methodennamen, Argumenten und Wirkungsbereich eine Nachricht. Die Nachrichtenschicht serialisiert sie, zerlegt sie in Pakete und versieht diese mit Steuerinformationen.

In der zweiten Phase überträgt die Steuereinheit die Pakete. Die Anzeigeeinheit bestätigt jedes empfangene Paket und setzt die Nachricht zusammen, sobald das letzte Paket eingetroffen ist. In der dritten Phase übergibt die Aufrufschicht die zusammengesetzte Nachricht dem Dispatcher, der die zugehörige Methode auf dem Adapter aufruft. Das Ergebnis wird wiederum zu einer Nachricht verpackt und zur Abholung bereitgestellt.

In der letzten Phase fragt die Steuereinheit ab, ob das Ergebnis bereitsteht, und holt es anschließend Paket für Paket ab. Die Aufrufschicht setzt die Antwort zusammen, liest die Nutzlast aus und gibt das Ergebnis an den Stellvertreter zurück, der es an die Anwendung weiterreicht.

Bemerkenswert an diesem Ablauf ist das Verhältnis von Nutzlast und Steuerverkehr. Ein einzelner Aufruf zerfällt in eine Folge von Paketen, die jeweils einzeln bestätigt werden, gefolgt von wiederholten Abfragen nach dem Ergebnis. Die Anzahl der erforderlichen Übertragungen und nicht die Geschwindigkeit des Übertragungswegs bestimmt damit die Dauer eines Aufrufs. Die Evaluation greift diese Beobachtung auf und beziffert sie.

6 Kommunikations- und Transportschicht

6.1 Motivation und Randbedingungen

6.2 Interceptor-Abstraktion

6.3 Anfragegetriebener Rückkanal

6.4 Frame-basiertes Nachrichtenprotokoll

6.4.1 Frame-Struktur

6.4.2 Nutzlastlänge und Formatunabhängigkeit

6.4.3 Kostenmodell der Rahmung

6.5 Fragmentierung und Rekonstruktion

6.6 Serialisierung der Nutzdaten

6.7 Zuverlässigkeitsmechanismen

7 Abstraktion des Displays und Objektmodell

7.1 Anforderungen an die Display-Abstraktion

7.2 Konzept der Remote-Objekte

7.3 Abbildung grafischer Elemente

7.4 Objekterzeugung, Referenzen und Lebenszyklus

7.5 Methodenaufrufe und Ereignisbehandlung

7.6 Designentscheidungen

8 Hardwareaufbau

8.1 Aufbau und Komponenten

8.2 Adapterplatine

8.2.1 Signalzuordnung und Pinbelegung

8.2.2 Aufbau und Inbetriebnahme

8.3 Integrierte Traegerplatine

9 Referenzimplementierung

9.1 Laufzeitumgebung

9.2 Anbindung des Displays

9.3 Bindung an den Bus

9.4 Datenstrukturen

9.4.1 Frame

9.4.2 Message

9.4.3 Payload

9.5 Ablauf eines vollständigen Aufrufs

9.6 Nebenläufigkeit und Fehlerbehandlung

10 Evaluation

10.1 Funktionaler Nachweis

10.2 Overhead durch Rahmung

10.3 Latenz und Anzahl der Übertragungen

10.4 Ressourcenverbrauch

10.5 Bewertung des Hardwareaufbaus

10.6 Diskussion der Ergebnisse

11 Diskussion und Übertragbarkeit

11.1 Limitierungen

11.1.1 Kommunikationsmodell

11.1.2 Latenz und Durchsatz

11.1.3 Abstraktionsgrenzen

11.2 Übertragbarkeit auf andere Verbindungsklassen

11.3 Mögliche Erweiterungen

11.3.1 Ereignisgetriebener Rückkanal

11.3.2 Alternative Serialisierung

11.3.3 Weitere Peripherieklassen

12 Fazit

13 Abbildungs- und Tabellenverzeichnis

Abbildungsverzeichnis

Abbildung 1 Schichtenmodell der Architektur. Beide Seiten sind gleich aufgebaut und
verkehren jeweils nur mit der unmittelbar darunterliegenden Schicht. 11

Tabellenverzeichnis

Tabelle I Vergleich der betrachteten Ansätze entlang der Auswahlkriterien 7

Code-Listings

Prompt-Listings

14 Abkürzungsverzeichnis

CBOR – Concise Binary Object Representation

gRPC – gRPC Remote Procedure Call

JSON – JavaScript Object Notation

ONC RPC – Open Network Computing Remote Procedure Call

Bibliografie

- [1] „CICS Transaction Server for z/OS“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://www.ibm.com/docs/en/cics-ts/5.5.0?topic=rpc-onc-concepts>
- [2] S. Microsystems, „RPC: Remote Procedure Call Protocol Specification“, Request for Comments RFC1050, Apr. 1988. doi: [10.17487/RFC1050](https://doi.org/10.17487/RFC1050).
- [3] R. Thurlow, „RPC: Remote Procedure Call Protocol Specification Version 2“, Request for Comments RFC5531, Mai 2009. doi: [10.17487/RFC5531](https://doi.org/10.17487/RFC5531).
- [4] „JSON-RPC 2.0 Specification“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://www.jsonrpc.org/specification>
- [5] „gRPC“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://grpc.io/>
- [6] T. Bray, „The JavaScript Object Notation (JSON) Data Interchange Format“, Request for Comments RFC8259, Dez. 2017. doi: [10.17487/RFC8259](https://doi.org/10.17487/RFC8259).
- [7] „MessagePack: It's like JSON. but Fast and Small.“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://msgpack.org/>
- [8] C. Bormann und P. E. Hoffman, „Concise Binary Object Representation (CBOR)“, Request for Comments RFC8949, Dez. 2020. doi: [10.17487/RFC8949](https://doi.org/10.17487/RFC8949).
- [9] „CBOR — Concise Binary Object Representation | Overview“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://cbor.io/>
- [10] „Protocol Buffers“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://protobuf.dev/>
- [11] „Nanopb — Zephyr Project Documentation“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://docs.zephyrproject.org/latest/services/serialization/nanopb.html>
- [12] „Firmata/Arduino“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://github.com/firmata/arduino>
- [13] „MQTT - The Standard for IoT Messaging“. Zugegriffen: 4. August 2026. [Online]. Verfügbar unter: <https://mqtt.org/>
- [14] „PUPRemote Documentation — Anton's Mindstorms Docs Documentation“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://docs.antonsmindstorms.com/en/latest/Software/PUPRemote/docs/index.html>
- [15] „Handlungsorientiertes Lernen“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://education.lego.com/de-de/>
- [16] „STM32F413/423 - STMicroelectronics“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://www.st.com/en/microcontrollers-microprocessors/stm32f413-423.html>
- [17] „Lego::Lab Karlsruhe“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://legolab-karlsruhe.de/mindstorms/spikeprime>
- [18] „Mct.de: SPI - Serial Peripheral Interface“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://web.archive.org/web/20190116220910/http://www.mct.de/faq/spi.html>
- [19] „I2C Bus“. Zugegriffen: 7. August 2026. [Online]. Verfügbar unter: <https://de.i2c-bus.org/>